Chapter 2. Creating and Populating a Database

This chapter provides you with the information you need to create your first database and to create the tables and associated data used for the examples in this book. You will also learn about various data types and see how to create tables using them. Because the examples in this book are executed against a MySQL database, this chapter is somewhat skewed toward MySQL's features and syntax, but most concepts are applicable to any server.

Creating a MySQL Database

If you want the ability to experiment with the data used for the examples in this book, you have two options:

- Download and install the MySQL server version 8.0 (or later) and load the Sakila example database from *https://dev.mysql.com/doc/index-other.html*.
- Go to *https://learning.oreilly.com/scenarios/mysql-sandbox/9781492079705* to access the MySQL Sandbox, which has the Sakila Sample database loaded in a MySQL instance. If you don't have one already, you'll have to set up an O'Reilly account. Then, click on the Start button and begin.

If you choose the second option, once you start the scenario, a MySQL server is installed and started, and then the Sakila schema and data are loaded. When it's ready, a standard `mysql>` prompt appears, and you can then start querying the sample database. This is certainly the easiest option, and I anticipate that most readers will choose this option; if this sounds good to you, feel free to skip ahead to the next section.

If you prefer to have your own copy of the data and want any changes you have made to be permanent, or if you are just interested in installing the MySQL server on your own machine, you may prefer the first option. You may also opt to use a MySQL server hosted in an environment such as Amazon Web Services or Google Cloud. In either case, you will need to perform the installation/configuration yourself, as it is beyond the scope of this book. Once your database is available, you will need to follow a few steps to load the Sakila sample database.

First, you will need to launch the `mysql` command-line client and provide a password, and then perform the following steps:

1. Go to *https://dev.mysql.com/doc/index-other.html* and download the files for "sakila database" under the Example Databases section.
2. Put the files in a local directory such as *C:\temp\sakila-db* (used for the next two steps, but overwrite with your directory path).
3. Type `source c:\temp\sakila-db\sakila-schema.sql` and press Enter.
4. Type `source c:\temp\sakila-db\sakila-data.sql` and press Enter.

You should now have a working database populated with all the data needed for the examples in this book.

NOTE

The Sakila sample database is made available by MySQL and is licensed via the New BSD license. Sakila contains data for a fictitious movie rental company, and includes tables such as store, inventory, film, customer, and payment. While actual movie rental stores are largely a thing of the past, with a little imagination we could rebrand it as a movie-streaming company by ignoring the `staff` and `address` tables and renaming `store` to `streaming_service`. However, the examples in this book will stick to the original script (pun intended).

Using the mysql Command-Line Tool

Unless you are using a temporary database session (the second option in the previous section), you will need to start the `mysql` command-line tool in order to interact with the database. To do so, you will need to open a Windows or Unix shell and execute the `mysql` utility. For example, if you are logging in using the root account, you would do the following:

```
mysql -u root -p;
```

You will then be asked for your password, after which you will see the `mysql>` prompt. To see all of the available databases, you can use the following command:

```
mysql> show databases;
+--------------------+
| Database           |
```

```
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
+--------------------+
5 rows in set (0.01 sec)
```

Since you will be using the Sakila database, you will need to specify the database you want to work with via the `use` command:

```
mysql> use sakila;
Database changed
```

Whenever you invoke the `mysql` command-line tool, you can specify both the username and database to use, as in the following:

```
mysql -u root -p sakila;
```

This will save you from having to type `use sakila;` every time you start up the tool. Now that you have established a session and specified the database, you will be able to issue SQL statements and view the results. For example, if you want to know the current date and time, you could issue the following query:

```
mysql> SELECT now();
+---------------------+
| now()               |
+---------------------+
| 2019-04-04 20:44:26 |
+---------------------+
1 row in set (0.01 sec)
```

The `now()` function is a built-in MySQL function that returns the current date and time. As you can see, the `mysql` command-line tool formats the results of your queries within a rectangle bounded by +, -, and | characters. After the results have been exhausted (in this case, there is only a single row of results), the `mysql` command-line tool shows how many rows were returned, along with how long the SQL statement took to execute.

ABOUT MISSING FROM CLAUSES

With some database servers, you won't be able to issue a query without a `from` clause that names at least one table. Oracle Database is a commonly used server for which this is true. For cases when you only need to call a function, Oracle provides a table called `dual`, which consists of a single column called `dummy` that contains a single row of data. In order to be compatible with Oracle Database, MySQL also provides a `dual` table. The previous query to determine the current date and time could therefore be written as:

```
mysql> SELECT now()
        FROM dual;
+---------------------+
| now()               |
+---------------------+
| 2019-04-04 20:44:26 |
+---------------------+
1 row in set (0.01 sec)
```

If you are not using Oracle and have no need to be compatible with Oracle, you can ignore the `dual` table altogether and use just a `select` clause without a `from` clause.

When you are done with the `mysql` command-line tool, simply type `quit;` or `exit;` to return to the Unix or Windows command shell.

MySQL Data Types

In general, all the popular database servers have the capacity to store the same types of data, such as strings, dates, and numbers. Where they typically differ is in the specialty data types, such as XML and JSON documents or spatial data. Since this is an introductory book on SQL and since 98% of the columns you encounter will be simple data types, this chapter covers only the character, date (a.k.a. temporal), and numeric data types. The use of SQL to query JSON documents will be explored in Chapter 18.

Character Data

Character data can be stored as either fixed-length or variable-length strings; the difference is that fixed-length strings are right-padded with spaces and always consume the same number of bytes, and variable-length strings are not right-padded with spaces and

don't always consume the same number of bytes. When defining a character column, you must specify the maximum size of any string to be stored in the column. For example, if you want to store strings up to 20 characters in length, you could use either of the following definitions:

```
char(20)    /* fixed-length */
varchar(20) /* variable-length */
```

The maximum length for `char` columns is currently 255 bytes, whereas `varchar` columns can be up to 65,535 bytes. If you need to store longer strings (such as emails, XML documents, etc.), then you will want to use one of the text types (`mediumtext` and `longtext`), which I cover later in this section. In general, you should use the `char` type when all strings to be stored in the column are of the same length, such as state abbreviations, and the `varchar` type when strings to be stored in the column are of varying lengths. Both `char` and `varchar` are used in a similar fashion in all the major database servers.

NOTE

An exception is made in the use of `varchar` for Oracle Database. Oracle users should use the `varchar2` type when defining variable-length character columns.

Character sets

For languages that use the Latin alphabet, such as English, there is a sufficiently small number of characters such that only a single byte is needed to store each character. Other languages, such as Japanese and Korean, contain large numbers of characters, thus requiring multiple bytes of storage for each character. Such character sets are therefore called *multibyte character sets*.

MySQL can store data using various character sets, both single- and multibyte. To view the supported character sets in your server, you can use the `show` command, as shown in the following example:

```
mysql> SHOW CHARACTER SET;
+----------+---------------------------------+--------------------+--------+
| Charset  | Description                     | Default collation  | Maxlen |
+----------+---------------------------------+--------------------+--------+
| armscii8 | ARMSCII-8 Armenian              | armscii8_general_ci |      1 |
```

```
| ascii    | US ASCII                        | ascii_general_ci    |      1 |
| big5     | Big5 Traditional Chinese        | big5_chinese_ci     |      2 |
| binary   | Binary pseudo charset           | binary              |      1 |
| cp1250   | Windows Central European        | cp1250_general_ci   |      1 |
| cp1251   | Windows Cyrillic                | cp1251_general_ci   |      1 |
| cp1256   | Windows Arabic                  | cp1256_general_ci   |      1 |
| cp1257   | Windows Baltic                  | cp1257_general_ci   |      1 |
| cp850    | DOS West European               | cp850_general_ci    |      1 |
| cp852    | DOS Central European            | cp852_general_ci    |      1 |
| cp866    | DOS Russian                     | cp866_general_ci    |      1 |
| cp932    | SJIS for Windows Japanese       | cp932_japanese_ci   |      2 |
| dec8     | DEC West European               | dec8_swedish_ci     |      1 |
| eucjpms  | UJIS for Windows Japanese       | eucjpms_japanese_ci |      3 |
| euckr    | EUC-KR Korean                   | euckr_korean_ci     |      2 |
| gb18030  | China National Standard GB18030 | gb18030_chinese_ci  |      4 |
| gb2312   | GB2312 Simplified Chinese       | gb2312_chinese_ci   |      2 |
| gbk      | GBK Simplified Chinese          | gbk_chinese_ci      |      2 |
| geostd8  | GEOSTD8 Georgian                | geostd8_general_ci  |      1 |
| greek    | ISO 8859-7 Greek                | greek_general_ci    |      1 |
| hebrew   | ISO 8859-8 Hebrew               | hebrew_general_ci   |      1 |
| hp8      | HP West European                | hp8_english_ci      |      1 |
| keybcs2  | DOS Kamenicky Czech-Slovak      | keybcs2_general_ci  |      1 |
| koi8r    | KOI8-R Relcom Russian           | koi8r_general_ci    |      1 |
| koi8u    | KOI8-U Ukrainian                | koi8u_general_ci    |      1 |
| latin1   | cp1252 West European            | latin1_swedish_ci   |      1 |
| latin2   | ISO 8859-2 Central European     | latin2_general_ci   |      1 |
| latin5   | ISO 8859-9 Turkish              | latin5_turkish_ci   |      1 |
| latin7   | ISO 8859-13 Baltic              | latin7_general_ci   |      1 |
| macce    | Mac Central European            | macce_general_ci    |      1 |
| macroman | Mac West European               | macroman_general_ci |      1 |
| sjis     | Shift-JIS Japanese              | sjis_japanese_ci    |      2 |
| swe7     | 7bit Swedish                    | swe7_swedish_ci     |      1 |
| tis620   | TIS620 Thai                     | tis620_thai_ci      |      1 |
| ucs2     | UCS-2 Unicode                   | ucs2_general_ci     |      2 |
| ujis     | EUC-JP Japanese                 | ujis_japanese_ci    |      3 |
| utf16    | UTF-16 Unicode                  | utf16_general_ci    |      4 |
| utf16le  | UTF-16LE Unicode                | utf16le_general_ci  |      4 |
| utf32    | UTF-32 Unicode                  | utf32_general_ci    |      4 |
| utf8     | UTF-8 Unicode                   | utf8_general_ci     |      3 |
| utf8mb4  | UTF-8 Unicode                   | utf8mb4_0900_ai_ci  |      4 |
+----------+---------------------------------+---------------------+--------+
41 rows in set (0.04 sec)
```

If the value in the fourth column, `maxlen`, is greater than 1, then the character set is a multibyte character set.

In prior versions of the MySQL server, the `latin1` character set was automatically chosen as the default character set, but version 8 defaults to `utf8mb4`. However, you may choose to use a different character set for each character column in your database, and you can even store different character sets within the same table. To choose a character set other than the default when defining a column, simply name one of the supported character sets after the type definition, as in:

```
varchar(20) character set latin1
```

With MySQL, you may also set the default character set for your entire database:

```
create database european_sales character set latin1;
```

While this is as much information regarding character sets as is appropriate for an introductory book, there is a great deal more to the topic of internationalization than what is shown here. If you plan to deal with multiple or unfamiliar character sets, you may want to pick up a book such as Jukka Korpela's *Unicode Explained: Internationalize Documents, Programs, and Web Sites* (O'Reilly).

Text data

If you need to store data that might exceed the 64 KB limit for `varchar` columns, you will need to use one of the text types.

Table 2-1 shows the available text types and their maximum sizes.

Table 2-1. MySQL text types

Text type	Maximum number of bytes
`tinytext`	255
`text`	65,535
`mediumtext`	16,777,215
`longtext`	4,294,967,295

When choosing to use one of the text types, you should be aware of the following:

- If the data being loaded into a text column exceeds the maximum size for that type, the data will be truncated.
- Trailing spaces will not be removed when data is loaded into the column.
- When using `text` columns for sorting or grouping, only the first 1,024 bytes are used, although this limit may be increased if necessary.
- The different text types are unique to MySQL. SQL Server has a single `text` type for large character data, whereas DB2 and Oracle use a data type called `clob`, for Character Large Object.
- Now that MySQL allows up to 65,535 bytes for `varchar` columns (it was limited to 255 bytes in version 4), there isn't any particular need to use the `tinytext` or `text` type.

If you are creating a column for free-form data entry, such as a `notes` column to hold data about customer interactions with your company's customer service department, then `varchar` will probably be adequate. If you are storing documents, however, you should choose either the `mediumtext` or `longtext` type.

NOTE

Oracle Database allows up to 2,000 bytes for `char` columns and 4,000 bytes for `varchar2` columns. For larger documents you may use the `clob` type. SQL Server can handle up to 8,000 bytes for both `char` and `varchar` data, but you can store up to 2 GB of data in a column defined as `varchar(max)`.

Numeric Data

Although it might seem reasonable to have a single numeric data type called "numeric," there are actually several different numeric data types that reflect the various ways in which numbers are used, as illustrated here:

A column indicating whether a customer order has been shipped

This type of column, referred to as a *Boolean*, would contain a `0` to indicate `false` and a `1` to indicate `true`.

A system-generated primary key for a transaction table

This data would generally start at `1` and increase in increments of one up to a potentially very large number.

An item number for a customer's electronic shopping basket

The values for this type of column would be positive whole numbers between 1 and, perhaps, 200 (for shopaholics).

Positional data for a circuit board drill machine

High-precision scientific or manufacturing data often requires accuracy to eight decimal points.

To handle these types of data (and more), MySQL has several different numeric data types. The most commonly used numeric types are those used to store whole numbers, or *integers*. When specifying one of these types, you may also specify that the data is *unsigned*, which tells the server that all data stored in the column will be greater than or equal to zero. Table 2-2 shows the five different data types used to store whole-number integers.

Table 2-2. MySQL integer types

Type	Signed range	Unsigned range
`tinyint`	–128 to 127	0 to 255
`smallint`	–32,768 to 32,767	0 to 65,535
`mediumint`	–8,388,608 to 8,388,607	0 to 16,777,215
`int`	–2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
`bigint`	–2^63 to 2^63 - 1	0 to 2^64 - 1

When you create a column using one of the integer types, MySQL will allocate an appropriate amount of space to store the data, which ranges from one byte for a `tinyint` to eight bytes for a `bigint`. Therefore, you should try to choose a type that will be large enough to hold the biggest number you can envision being stored in the column without needlessly wasting storage space.

For floating-point numbers (such as 3.1415927), you may choose from the numeric types shown in Table 2-3.

Table 2-3. MySQL floating-point types

Type	Numeric range
`float(p,s)`	–3.402823466E+38 to –1.175494351E-38 and 1.175494351E-38 to 3.402823466E+38
double(*p*, *s*)	–1.7976931348623157E+308 to –2.2250738585072014E-308 and 2.2250738585072014E-308 to 1.7976931348623157E+308

When using a floating-point type, you can specify a *precision* (the total number of allowable digits both to the left and to the right of the decimal point) and a *scale* (the number of allowable digits to the right of the decimal point), but they are not required. These values are represented in Table 2-3 as *p* and *s*. If you specify a precision and scale for your floating-point column, remember that the data stored in the column will be rounded if the number of digits exceeds the scale and/or precision of the column. For example, a column defined as `float(4,2)` will store a total of four digits, two to the left of the decimal and two to the right of the decimal. Therefore, such a column would handle the numbers 27.44 and 8.19 just fine, but the number 17.8675 would be rounded to 17.87, and attempting to store the number 178.375 in your `float(4,2)` column would generate an error.

Like the integer types, floating-point columns can be defined as `unsigned`, but this designation only prevents negative numbers from being stored in the column rather than altering the range of data that may be stored in the column.

Temporal Data

Along with strings and numbers, you will almost certainly be working with information about dates and/or times. This type of data is referred to as *temporal*, and some examples of temporal data in a database include:

- The future date that a particular event is expected to happen, such as shipping a customer's order
- The date that a customer's order was shipped
- The date and time that a user modified a particular row in a table
- An employee's birth date
- The year corresponding to a row in a `yearly_sales` fact table in a data warehouse

- The elapsed time needed to complete a wiring harness on an automobile assembly line

MySQL includes data types to handle all of these situations. Table 2-4 shows the temporal data types supported by MySQL.

Table 2-4. MySQL temporal types

Type	Default format	Allowable values
`date`	YYYY-MM-DD	`1000-01-01` to `9999-12-31`
datetime	YYYY-MM-DD HH:MI:SS	1000-01-01 00:00:00.000000 to 9999-12-31 23:59:59.999999
`timestamp`	YYYY-MM-DD HH:MI:SS	`1970-01-01 00:00:00.000000` to `2038-01-18 22:14:07.999999`
year	YYYY	1901 to 2155
`time`	HHH:MI:SS	`–838:59:59.000000` to `838:59:59.000000`

While database servers store temporal data in various ways, the purpose of a format string (second column of Table 2-4) is to show how the data will be represented when retrieved, along with how a date string should be constructed when inserting or updating a temporal column. Thus, if you wanted to insert the date March 23, 2020, into a `date` column using the default format `YYYY-MM-DD`, you would use the string `'2020-03-23'`. Chapter 7 fully explores how temporal data is constructed and displayed.

The `datetime`, `timestamp`, and `time` types also allow fractional seconds of up to 6 decimal places (microseconds). When defining columns using one of these data types, you may supply a value from 0 to 6; for example, specifying `datetime(2)` would allow your time values to include hundredths of a second.

NOTE

Each database server allows a different range of dates for temporal columns. Oracle Database accepts dates ranging from 4712 BC to 9999 AD, while SQL Server only handles dates ranging from 1753 AD to 9999 AD (unless you are using SQL Server 2008's `datetime2` data type, which allows for dates ranging from 1 AD to 9999 AD). MySQL falls in between Oracle and SQL Server and can store dates from 1000 AD to 9999 AD. Although this might not make any difference for most systems that track current and future events, it is important to keep in mind if you are storing historical dates.

Table 2-5 describes the various components of the date formats shown in Table 2-4.

Table 2-5. Date format components

Component	Definition	Range
YYYY	Year, including century	`1000` to `9999`
MM	Month	01 (January) to 12 (December)
DD	Day	`01` to `31`
HH	Hour	00 to 23
HHH	Hours (elapsed)	`-838` to `838`
MI	Minute	00 to 59
SS	Second	`00` to `59`

Here's how the various temporal types would be used to implement the examples shown earlier:

- Columns to hold the expected future shipping date of a customer order and an employee's birth date would use the `date` type, since it is unrealistic to schedule a future shipment down to the second and unnecessary to know at what time a person was born.
- A column to hold information about when a customer order was actually shipped would use the `datetime` type, since it is important to track not only the date that the shipment occurred but the time as well.

- A column that tracks when a user last modified a particular row in a table would use the `timestamp` type. The `timestamp` type holds the same information as the `datetime` type (year, month, day, hour, minute, second), but a `timestamp` column will automatically be populated with the current date/time by the MySQL server when a row is added to a table or when a row is later modified.
- A column holding just year data would use the `year` type.
- Columns that hold data regarding the length of time needed to complete a task would use the `time` type. For this type of data, it would be unnecessary and confusing to store a date component, since you are interested only in the number of hours/minutes/seconds needed to complete the task. This information could be derived using two `datetime` columns (one for the task start date/time and the other for the task completion date/time) and subtracting one from the other, but it is simpler to use a single `time` column.

Chapter 7 explores how to work with each of these temporal data types.

Table Creation

Now that you have a firm grasp on what data types may be stored in a MySQL database, it's time to see how to use these types in table definitions. Let's start by defining a table to hold information about a person.

Step 1: Design

A good way to start designing a table is to do a bit of brainstorming to see what kind of information would be helpful to include. Here's what I came up with after thinking for a short time about the types of information that describe a person:

- Name
- Eye color
- Birth date
- Address
- Favorite foods

This is certainly not an exhaustive list, but it's good enough for now. The next step is to assign column names and data types. Table 2-6 shows my initial attempt.

Table 2-6. Person table, first pass

Column	Type	Allowable values
name	varchar(40)	
eye_color	char(2)	BL , BR , GR
birth_date	date	
address	varchar(100)	
favorite_foods	varchar(200)	

The `name`, `address`, and `favorite_foods` columns are of type `varchar` and allow for free-form data entry. The `eye_color` column allows two characters that should equal only `BR`, `BL`, or `GR`. The `birth_date` column is of type `date`, since a time component is not needed.

Step 2: Refinement

In Chapter 1, you were introduced to the concept of *normalization*, which is the process of ensuring that there are no duplicate (other than foreign keys) or compound columns in your database design. In looking at the columns in the `person` table a second time, the following issues arise:

- The `name` column is actually a compound object consisting of a first name and a last name.
- Since multiple people can have the same name, eye color, birth date, and so forth, there are no columns in the `person` table that guarantee uniqueness.
- The `address` column is also a compound object consisting of street, city, state/province, country, and postal code.
- The `favorite_foods` column is a list containing zero, one, or more independent items. It would be best to create a separate table for this data that includes a foreign key to the `person` table so that you know to which person a particular food may be attributed.

After taking these issues into consideration, Table 2-7 gives a normalized version of the `person` table.

Table 2-7. Person table, second pass

Column	Type	Allowable values
person_id	smallint (unsigned)	
first_name	varchar(20)	
last_name	varchar(20)	
eye_color	char(2)	BR , BL , GR
birth_date	date	
street	varchar(30)	
city	varchar(20)	
state	varchar(20)	
country	varchar(20)	
postal_code	varchar(20)	

Now that the person table has a primary key (person_id) to guarantee uniqueness, the next step is to build a favorite_food table that includes a foreign key to the person table. Table 2-8 shows the result.

Table 2-8. favorite_food table

Column	Type
person_id	smallint (unsigned)
food	varchar(20)

The person_id and food columns comprise the primary key of the favorite_food table, and the person_id column is also a foreign key to the person table.

HOW MUCH IS ENOUGH?

Moving the `favorite_foods` column out of the `person` table was definitely a good idea, but are we done yet? What happens, for example, if one person lists "pasta" as a favorite food while another person lists "spaghetti"? Are they the same thing? In order to prevent this problem, you might decide that you want people to choose their favorite foods from a list of options, in which case you should create a `food` table with `food_id` and `food_name` columns and then change the `favorite_food` table to contain a foreign key to the `food` table. While this design would be fully normalized, you might decide that you simply want to store the values that the user has entered, in which case you may leave the table as is.

Step 3: Building SQL Schema Statements

Now that the design is complete for the two tables holding information about people and their favorite foods, the next step is to generate SQL statements to create the tables in the database. Here is the statement to create the `person` table:

```
CREATE TABLE person
 (person_id SMALLINT UNSIGNED,
  fname VARCHAR(20),
  lname VARCHAR(20),
  eye_color CHAR(2),
  birth_date DATE,
  street VARCHAR(30),
  city VARCHAR(20),
  state VARCHAR(20),
  country VARCHAR(20),
  postal_code VARCHAR(20),
  CONSTRAINT pk_person PRIMARY KEY (person_id)
 );
```

Everything in this statement should be fairly self-explanatory except for the last item; when you define your table, you need to tell the database server what column or columns will serve as the primary key for the table. You do this by creating a *constraint* on the table. You can add several types of constraints to a table definition. This constraint is a *primary key constraint*. It is created on the `person_id` column and given the name `pk_person`.

While on the topic of constraints, there is another type of constraint that would be useful for the `person` table. In Table 2-6, I added a third column to show the allowable values for certain columns (such as `'BR'` and `'BL'` for the `eye_color` column). Another type of constraint called a *check constraint* constrains the allowable values for a particular column. MySQL allows a check constraint to be attached to a column definition, as in the following:

```
eye_color CHAR(2) CHECK (eye_color IN ('BR','BL','GR')),
```

While check constraints operate as expected on most database servers, the MySQL server allows check constraints to be defined but does not enforce them. However, MySQL does provide another character data type called `enum` that merges the check constraint into the data type definition. Here's what it would look like for the `eye_color` column definition:

```
eye_color ENUM('BR','BL','GR'),
```

Here's how the `person` table definition looks with an `enum` data type for the `eye_color` column:

```
CREATE TABLE person
 (person_id SMALLINT UNSIGNED,
  fname VARCHAR(20),
  lname VARCHAR(20),
  eye_color ENUM('BR','BL','GR'),
  birth_date DATE,
  street VARCHAR(30),
  city VARCHAR(20),
  state VARCHAR(20),
  country VARCHAR(20),
  postal_code VARCHAR(20),
  CONSTRAINT pk_person PRIMARY KEY (person_id)
 );
```

Later in this chapter, you will see what happens if you try to add data to a column that violates its check constraint (or, in the case of MySQL, its enumeration values).

You are now ready to run the `create table` statement using the `mysql` command-line tool. Here's what it looks like:

```
mysql> CREATE TABLE person
    -> (person_id SMALLINT UNSIGNED,
    ->  fname VARCHAR(20),
    ->  lname VARCHAR(20),
    ->  eye_color ENUM('BR','BL','GR'),
    ->  birth_date DATE,
    ->  street VARCHAR(30),
    ->  city VARCHAR(20),
    ->  state VARCHAR(20),
    ->  country VARCHAR(20),
    ->  postal_code VARCHAR(20),
    ->  CONSTRAINT pk_person PRIMARY KEY (person_id)
    -> );
Query OK, 0 rows affected (0.37 sec)
```

After processing the `create table` statement, the MySQL server returns the message "Query OK, 0 rows affected," which tells me that the statement had no syntax errors.

If you want to make sure that the `person` table does, in fact, exist, you can use the `describe` command (or `desc` for short) to look at the table definition:

```
mysql> desc person;
+-------------+----------------------+------+-----+---------+-------+
| Field       | Type                 | Null | Key | Default | Extra |
+-------------+----------------------+------+-----+---------+-------+
| person_id   | smallint(5) unsigned | NO   | PRI | NULL    |       |
| fname       | varchar(20)          | YES  |     | NULL    |       |
| lname       | varchar(20)          | YES  |     | NULL    |       |
| eye_color   | enum('BR','BL','GR') | YES  |     | NULL    |       |
| birth_date  | date                 | YES  |     | NULL    |       |
| street      | varchar(30)          | YES  |     | NULL    |       |
| city        | varchar(20)          | YES  |     | NULL    |       |
| state       | varchar(20)          | YES  |     | NULL    |       |
| country     | varchar(20)          | YES  |     | NULL    |       |
| postal_code | varchar(20)          | YES  |     | NULL    |       |
+-------------+----------------------+------+-----+---------+-------+
10 rows in set (0.00 sec)
```

Columns 1 and 2 of the `describe` output are self-explanatory. Column 3 shows whether a particular column can be omitted when data is inserted into the table. I purposefully left this topic out of the discussion for now (see the following sidebar for a short discourse), but we explore it fully in Chapter 4. The fourth column shows whether a column takes

part in any keys (primary or foreign); in this case, the `person_id` column is marked as the primary key. Column 5 shows whether a particular column will be populated with a default value if you omit the column when inserting data into the table. The sixth column (called "Extra") shows any other pertinent information that might apply to a column.

WHAT IS NULL?

In some cases, it is not possible or applicable to provide a value for a particular column in your table. For example, when adding data about a new customer order, the `ship_date` column cannot yet be determined. In this case, the column is said to be *null* (note that I do not say that it *equals* null), which indicates the absence of a value. Null is used for various cases where a value cannot be supplied, such as:

- Not applicable
- Unknown
- Empty set

When designing a table, you may specify which columns are allowed to be null (the default) and which columns are not allowed to be null (designated by adding the keywords `not null` after the type definition).

Now that you have created the `person` table, your next step will be to then create the `favorite_food` table:

```
mysql> CREATE TABLE favorite_food
    -> (person_id SMALLINT UNSIGNED,
    -> food VARCHAR(20),
    -> CONSTRAINT pk_favorite_food PRIMARY KEY (person_id, food),
    -> CONSTRAINT fk_fav_food_person_id FOREIGN KEY (person_id)
    -> REFERENCES person (person_id)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

This should look very similar to the `create table` statement for the `person` table, with the following exceptions:

- Since a person can have more than one favorite food (which is the reason this table was created in the first place), it takes more than just the `person_id` column to

guarantee uniqueness in the table. This table, therefore, has a two-column primary key: `person_id` and `food`.

- The `favorite_food` table contains another type of constraint which is called a *foreign key constraint*. This constrains the values of the `person_id` column in the `favorite_food` table to include *only* values found in the `person` table. With this constraint in place, I will not be able to add a row to the `favorite_food` table indicating that `person_id 27` likes pizza if there isn't already a row in the `person` table having a `person_id` of `27`.

NOTE

If you forget to create the foreign key constraint when you first create the table, you can add it later via the `alter table` statement.

`describe` shows the following after executing the `create table` statement:

```
mysql> desc favorite_food;
+-----------+---------------------+------+-----+---------+-------+
| Field     | Type                | Null | Key | Default | Extra |
+-----------+---------------------+------+-----+---------+-------+
| person_id | smallint(5) unsigned | NO   | PRI | NULL    |       |
| food      | varchar(20)         | NO   | PRI | NULL    |       |
+-----------+---------------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

Now that the tables are in place, the next logical step is to add some data.

Populating and Modifying Tables

With the `person` and `favorite_food` tables in place, you can now begin to explore the four SQL data statements: `insert`, `update`, `delete`, and `select`.

Inserting Data

Since there is not yet any data in the `person` and `favorite_food` tables, the first of the four SQL data statements to be explored will be the `insert` statement. There are three main components to an `insert` statement:

- The name of the table into which to add the data
- The names of the columns in the table to be populated
- The values with which to populate the columns

You are not required to provide data for every column in the table (unless all the columns in the table have been defined as `not null`). In some cases, those columns that are not included in the initial `insert` statement will be given a value later via an `update` statement. In other cases, a column may never receive a value for a particular row of data (such as a customer order that is canceled before being shipped, thus rendering the `ship_date` column inapplicable).

Generating numeric key data

Before inserting data into the `person` table, it would be useful to discuss how values are generated for numeric primary keys. Other than picking a number out of thin air, you have a couple of options:

- Look at the largest value currently in the table and add one.
- Let the database server provide the value for you.

Although the first option may seem valid, it proves problematic in a multiuser environment, since two users might look at the table at the same time and generate the same value for the primary key. Instead, all database servers on the market today provide a safe, robust method for generating numeric keys. In some servers, such as the Oracle Database, a separate schema object is used (called a *sequence*); in the case of MySQL, however, you simply need to turn on the *auto-increment* feature for your primary key column. Normally, you would do this at table creation, but doing it now provides the opportunity to learn another SQL schema statement, `alter table`, which is used to modify the definition of an existing table:

```
ALTER TABLE person MODIFY person_id SMALLINT UNSIGNED AUTO_INCREMENT;
```

WARNING

If you are running these statements in your database, you will first need to disable the foreign key constraint on the `favorite_food` table, and then re-enable the constraints when finished. The progression of statements would be:

```
set foreign_key_checks=0;
ALTER TABLE person
   MODIFY person_id SMALLINT UNSIGNED AUTO_INCREMENT;
set foreign_key_checks=1;
```

This statement essentially redefines the `person_id` column in the `person` table. If you describe the table, you will now see the auto-increment feature listed under the "Extra" column for `person_id`:

```
mysql> DESC person;
+-------------+-----------------------+------+-----+---------+----------------+
| Field       | Type                  | Null | Key | Default | Extra          |
+-------------+-----------------------+------+-----+---------+----------------+
| person_id   | smallint(5) unsigned  | NO   | PRI | NULL    | auto_increment |
| .           |                       |      |     |         |                |
| .           |                       |      |     |         |                |
| .           |                       |      |     |         |                |
```

When you insert data into the `person` table, you simply provide a `null` value for the `person_id` column, and MySQL will populate the column with the next available number (by default, MySQL starts at 1 for auto-increment columns).

The insert statement

Now that all the pieces are in place, it's time to add some data. The following statement creates a row in the `person` table for William Turner:

```
mysql> INSERT INTO person
    ->   (person_id, fname, lname, eye_color, birth_date)
    -> VALUES (null, 'William','Turner', 'BR', '1972-05-27');
Query OK, 1 row affected (0.22 sec)
```

The feedback ("Query OK, 1 row affected") tells you that your statement syntax was proper and that one row was added to the database (since it was an `insert` statement). You can look at the data just added to the table by issuing a `select` statement:

```
mysql> SELECT person_id, fname, lname, birth_date
    -> FROM person;
+-----------+---------+--------+------------+
| person_id | fname   | lname  | birth_date |
+-----------+---------+--------+------------+
|         1 | William | Turner | 1972-05-27 |
+-----------+---------+--------+------------+
1 row in set (0.06 sec)
```

As you can see, the MySQL server generated a value of `1` for the primary key. Since there is only a single row in the `person` table, I neglected to specify which row I am interested in and simply retrieved all the rows in the table. If there were more than one row in the table, however, I could add a `where` clause to specify that I want to retrieve data only for the row having a value of `1` for the `person_id` column:

```
mysql> SELECT person_id, fname, lname, birth_date
    -> FROM person
    -> WHERE person_id = 1;
+-----------+---------+--------+------------+
| person_id | fname   | lname  | birth_date |
+-----------+---------+--------+------------+
|         1 | William | Turner | 1972-05-27 |
+-----------+---------+--------+------------+
1 row in set (0.00 sec)
```

While this query specifies a particular primary key value, you can use any column in the table to search for rows, as shown by the following query, which finds all rows with a value of `'Turner'` for the `lname` column:

```
mysql> SELECT person_id, fname, lname, birth_date
    -> FROM person
    -> WHERE lname = 'Turner';
+-----------+---------+--------+------------+
| person_id | fname   | lname  | birth_date |
+-----------+---------+--------+------------+
|         1 | William | Turner | 1972-05-27 |
```

```
+----------+---------+--------+------------+
1 row in set (0.00 sec)
```

Before moving on, a couple of things about the earlier `insert` statement are worth mentioning:

- Values were not provided for any of the address columns. This is fine, since `null`s are allowed for those columns.
- The value provided for the `birth_date` column was a string. As long as you match the required format shown in Table 2-4, MySQL will convert the string to a date for you.
- The column names and the values provided must correspond in number and type. If you name seven columns and provide only six values or if you provide values that cannot be converted to the appropriate data type for the corresponding column, you will receive an error.

William Turner has also provided information about his favorite three foods, so here are three `insert` statements to store his food preferences:

```
mysql> INSERT INTO favorite_food (person_id, food)
    -> VALUES (1, 'pizza');
Query OK, 1 row affected (0.01 sec)
mysql> INSERT INTO favorite_food (person_id, food)
    -> VALUES (1, 'cookies');
Query OK, 1 row affected (0.00 sec)
mysql> INSERT INTO favorite_food (person_id, food)
    -> VALUES (1, 'nachos');
Query OK, 1 row affected (0.01 sec)
```

Here's a query that retrieves William's favorite foods in alphabetical order using an `order by` clause:

```
mysql> SELECT food
    -> FROM favorite_food
    -> WHERE person_id = 1
    -> ORDER BY food;
+---------+
| food    |
+---------+
| cookies |
| nachos  |
```

```
| pizza   |
+---------+
3 rows in set (0.02 sec)
```

The `order by` clause tells the server how to sort the data returned by the query. Without the `order by` clause, there is no guarantee that the data in the table will be retrieved in any particular order.

So that William doesn't get lonely, you can execute another `insert` statement to add Susan Smith to the `person` table:

```
mysql> INSERT INTO person
    -> (person_id, fname, lname, eye_color, birth_date,
    -> street, city, state, country, postal_code)
    -> VALUES (null, 'Susan','Smith', 'BL', '1975-11-02',
    ->  '23 Maple St.', 'Arlington', 'VA', 'USA', '20220');
 Query OK, 1 row affected (0.01 sec)
```

Since Susan was kind enough to provide her address, we included five more columns than when William's data was inserted. If you query the table again, you will see that Susan's row has been assigned the value `2` for its primary key value:

```
mysql> SELECT person_id, fname, lname, birth_date
    -> FROM person;
+-----------+---------+--------+------------+
| person_id | fname   | lname  | birth_date |
+-----------+---------+--------+------------+
|         1 | William | Turner | 1972-05-27 |
|         2 | Susan   | Smith  | 1975-11-02 |
+-----------+---------+--------+------------+
2 rows in set (0.00 sec)
```

CAN I GET THAT IN XML?

If you will be working with XML data, you will be happy to know that most database servers provide a simple way to generate XML output from a query. With MySQL, for example, you can use the `--xml` option when invoking the `mysql` tool, and all your output will automatically be formatted using XML. Here's what the favorite-food data looks like as an XML document:

```
C:\database> mysql -u lrngsql -p --xml bank
Enter password: xxxxxx
Welcome to the MySQL Monitor...

Mysql> SELECT * FROM favorite_food;
<?xml version="1.0"?>

<resultset statement="select * from favorite_food"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <row>
      <field name="person_id">1</field>
      <field name="food">cookies</field>
  </row>
  <row>
      <field name="person_id">1</field>
      <field name="food">nachos</field>
  </row>
  <row>
      <field name="person_id">1</field>
      <field name="food">pizza</field>
  </row>
</resultset>
3 rows in set (0.00 sec)
```

With SQL Server, you don't need to configure your command-line tool; you just need to add the `for xml` clause to the end of your query, as in:

```
SELECT * FROM favorite_food
FOR XML AUTO, ELEMENTS
```

Updating Data

When the data for William Turner was initially added to the table, data for the various address columns was not included in the `insert` statement. The next statement shows how these columns can be populated at a later time via an `update` statement:

```
mysql> UPDATE person
    -> SET street = '1225 Tremont St.',
    ->   city = 'Boston',
    ->   state = 'MA',
    ->   country = 'USA',
    ->   postal_code = '02138'
    -> WHERE person_id = 1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

The server responded with a two-line message: the "Rows matched: 1" item tells you that the condition in the `where` clause matched a single row in the table, and the "Changed: 1" item tells you that a single row in the table has been modified. Since the `where` clause specifies the primary key of William's row, this is exactly what you would expect to have happen.

Depending on the conditions in your `where` clause, it is also possible to modify more than one row using a single statement. Consider, for example, what would happen if your `where` clause looked as follows:

```
WHERE person_id < 10
```

Since both William and Susan have a `person_id` value less than 10, both of their rows would be modified. If you leave off the `where` clause altogether, your `update` statement will modify every row in the table.

Deleting Data

It seems that William and Susan aren't getting along very well together, so one of them has got to go. Since William was there first, Susan will get the boot courtesy of the `delete` statement:

```
mysql> DELETE FROM person
    -> WHERE person_id = 2;
Query OK, 1 row affected (0.01 sec)
```

Again, the primary key is being used to isolate the row of interest, so a single row is deleted from the table. Like the `update` statement, more than one row can be deleted depending on the conditions in your `where` clause, and all rows will be deleted if the `where` clause is omitted.

When Good Statements Go Bad

So far, all of the SQL data statements shown in this chapter have been well formed and have played by the rules. Based on the table definitions for the `person` and `favorite_food` tables, however, there are lots of ways that you can run afoul when inserting or modifying data. This section shows you some of the common mistakes that you might come across and how the MySQL server will respond.

Nonunique Primary Key

Because the table definitions include the creation of primary key constraints, MySQL will make sure that duplicate key values are not inserted into the tables. The next statement attempts to bypass the auto-increment feature of the `person_id` column and create another row in the `person` table with a `person_id` of `1`:

```
mysql> INSERT INTO person
    ->  (person_id, fname, lname, eye_color, birth_date)
    -> VALUES (1, 'Charles','Fulton', 'GR', '1968-01-15');
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

There is nothing stopping you (with the current schema objects, at least) from creating two rows with identical names, addresses, birth dates, and so on, as long as they have different values for the `person_id` column.

Nonexistent Foreign Key

The table definition for the `favorite_food` table includes the creation of a foreign key constraint on the `person_id` column. This constraint ensures that all values of `person_id`

entered into the `favorite_food` table exist in the `person` table. Here's what would happen if you tried to create a row that violates this constraint:

```
mysql> INSERT INTO favorite_food (person_id, food)
    -> VALUES (999, 'lasagna');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint
fails ('sakila'.'favorite_food', CONSTRAINT 'fk_fav_food_person_id' FOREIGN KEY
('person_id') REFERENCES 'person' ('person_id'))
```

In this case, the `favorite_food` table is considered the *child* and the `person` table is considered the *parent*, since the `favorite_food` table is dependent on the `person` table for some of its data. If you plan to enter data into both tables, you will need to create a row in `parent` before you can enter data into `favorite_food`.

NOTE

Foreign key constraints are enforced only if your tables are created using the InnoDB storage engine. We discuss MySQL's storage engines in Chapter 12.

Column Value Violations

The `eye_color` column in the `person` table is restricted to the values `'BR'` for brown, `'BL'` for blue, and `'GR'` for green. If you mistakenly attempt to set the value of the column to any other value, you will receive the following response:

```
mysql> UPDATE person
    -> SET eye_color = 'ZZ'
    -> WHERE person_id = 1;
ERROR 1265 (01000): Data truncated for column 'eye_color' at row 1
```

The error message is a bit confusing, but it gives you the general idea that the server is unhappy about the value provided for the `eye_color` column.

Invalid Date Conversions

If you construct a string with which to populate a `date` column and that string does not match the expected format, you will receive another error. Here's an example that uses a date format that does not match the default date format of `YYYY-MM-DD`:

```
mysql> UPDATE person
    -> SET birth_date = 'DEC-21-1980'
    -> WHERE person_id = 1;
ERROR 1292 (22007): Incorrect date value: 'DEC-21-1980' for column 'birth_date'
at row 1
```

In general, it is always a good idea to explicitly specify the format string rather than relying on the default format. Here's another version of the statement that uses the `str_to_date` function to specify which format string to use:

```
mysql> UPDATE person
    -> SET birth_date = str_to_date('DEC-21-1980' , '%b-%d-%Y')
    -> WHERE person_id = 1;
Query OK, 1 row affected (0.12 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Not only is the database server happy, but William is happy as well (we just made him eight years younger, without the need for expensive cosmetic surgery!).

NOTE

Earlier in the chapter, when I discussed the various temporal data types, I showed date-formatting strings such as `YYYY-MM-DD`. While many database servers use this style of formatting, MySQL uses `%Y` to indicate a four-character year. Here are a few more formatters that you might need when converting strings to `datetime`s in MySQL:

```
%a The short weekday name, such as Sun, Mon, ...
%b The short month name, such as Jan, Feb, ...
%c The numeric month (0..12)
%d The numeric day of the month (00..31)
%f The number of microseconds (000000..999999)
%H The hour of the day, in 24-hour format (00..23)
%h The hour of the day, in 12-hour format (01..12)
%i The minutes within the hour (00..59)
%j The day of year (001..366)
%M The full month name (January..December)
%m The numeric month
%p AM or PM
%s The number of seconds (00..59)
%W The full weekday name (Sunday..Saturday)
%w The numeric day of the week (0=Sunday..6=Saturday)
%Y The four-digit year
```

The Sakila Database

For the remainder of the book, most examples will use a sample database called Sakila, which is made available by the nice people at MySQL. This database models a chain of DVD rental stores, which is a bit outdated, but with a bit of imagination it can be re-branded as a video-streaming company. Some of the tables include `customer`, `film`, `actor`, `payment`, `rental`, and `category`. The entire schema and example data should have been created when you followed the final steps at the beginning of the chapter for loading the MySQL server and generating the sample data. For a diagram of the tables and their columns and relationships, see Appendix A.

Table 2-9 shows some of the tables used in the Sakila schema, along with short definitions of each.

Table 2-9. Sakila schema definitions

Table name	Definition
`film`	A movie that has been released and can be rented
`actor`	A person who acts in films
`customer`	A person who watches films
`category`	A genre of films
`payment`	A rental of a film by a customer
`language`	A language spoken by the actors of a film
`film_actor`	An actor in a film
`inventory`	A film available for rental

Feel free to experiment with the tables as much as you want, including adding your own tables to expand the business functions. You can always drop the database and re-create it from the downloaded file if you want to make sure your sample data is intact. If you are using the temporary session, any changes you make will be lost when the session

closes, so you may want to keep a script of your changes so you can re-create any changes you have made.

If you want to see the tables available in your database, you can use the `show tables` command, as in:

```
mysql> show tables;
+----------------------------+
| Tables_in_sakila           |
+----------------------------+
| actor                      |
| actor_info                 |
| address                    |
| category                   |
| city                       |
| country                    |
| customer                   |
| customer_list              |
| film                       |
| film_actor                 |
| film_category              |
| film_list                  |
| film_text                  |
| inventory                  |
| language                   |
| nicer_but_slower_film_list |
| payment                    |
| rental                     |
| sales_by_film_category     |
| sales_by_store             |
| staff                      |
| staff_list                 |
| store                      |
+----------------------------+
23 rows in set (0.02 sec)
```

Along with the 23 tables in the Sakila schema, your table listing may also include the two tables created in this chapter: `person` and `favorite_food`. These tables will not be used in later chapters, so feel free to drop them by issuing the following set of commands:

```
mysql> DROP TABLE favorite_food;
Query OK, 0 rows affected (0.56 sec)
```

```
mysql> DROP TABLE person;
Query OK, 0 rows affected (0.05 sec)
```

If you want to look at the columns in a table, you can use the `describe` command. Here's an example of the `describe` output for the `customer` table:

```
mysql> desc customer;
+-------------+-------------+------+-----+-------------+----------------------+
| Field       | Type        | Null | Key | Default     | Extra                |
+-------------+-------------+------+-----+-------------+----------------------+
| customer_id | smallint(5)  | NO   | PRI | NULL        | auto_increment       |
              unsigned
| store_id    | tinyint(3)   | NO   | MUL | NULL        |                      |
              unsigned
| first_name  | varchar(45)  | NO   |     | NULL        |                      |
| last_name   | varchar(45)  | NO   | MUL | NULL        |                      |
| email       | varchar(50)  | YES  |     | NULL        |                      |
| address_id  | smallint(5)  | NO   | MUL | NULL        |                      |
              unsigned
| active      | tinyint(1)   | NO   |     | 1           |                      |
| create_date | datetime     | NO   |     | NULL        |                      |
| last_update | timestamp    | YES  |     | CURRENT_    | DEFAULT_GENERATED on
                                        TIMESTAMP     update CURRENT_
                                                  TIMESTAMP         |
+-------------+-------------+------+-----+-------------+----------------------+
```

The more comfortable you are with the example database, the better you will understand the examples and, consequently, the concepts in the following chapters.

Previous chapter
< 1. A Little Background

Next chapter
3. Query Primer >